



เว็บแอปพลิเคชันจัดการ โครงรายวิชาของสาขา วิทยาการคอมพิวเตอร์

รายวิชา CP353002

PRINCIPLES OF SOFTWARE DESIGN AND DEVELOPMENT

รายชื่อสมาชิกกลุ่ม

นางสาวณปภัช คุณนิธิภาณุ 643021223-3

นางสาวจุฬาลักษณ์ จันทรศรี 653380124-4

นายชลพัฒน์ ปิ่นมณี 653380126-0

นางสาวพลอยชมพู วงศ์กียรติกุล 653380141-4

นางสาววรัญญา อ่วมเมือง 653380281-8

การทำงานของระบบ

เว็บแอปพลิเคชันจัดการโปรเจกต์รายวิชาของสาขาวิทยาการคอมพิวเตอร์ ถูกออกแบบมาเพื่อเก็บรวบรวมโปรเจกต์ของนักศึกษาสาขาวิทยาการคอมพิวเตอร์ โดยสามารถจับเก็บโปรเจกต์ของตนเองไว้และยังสามารถค้นหาและดูตัวอย่างโปรเจกต์ที่สนใจในแต่ละหัวข้อได้สะดวกตามต้องการ

เทคโนโลยีที่ใช้

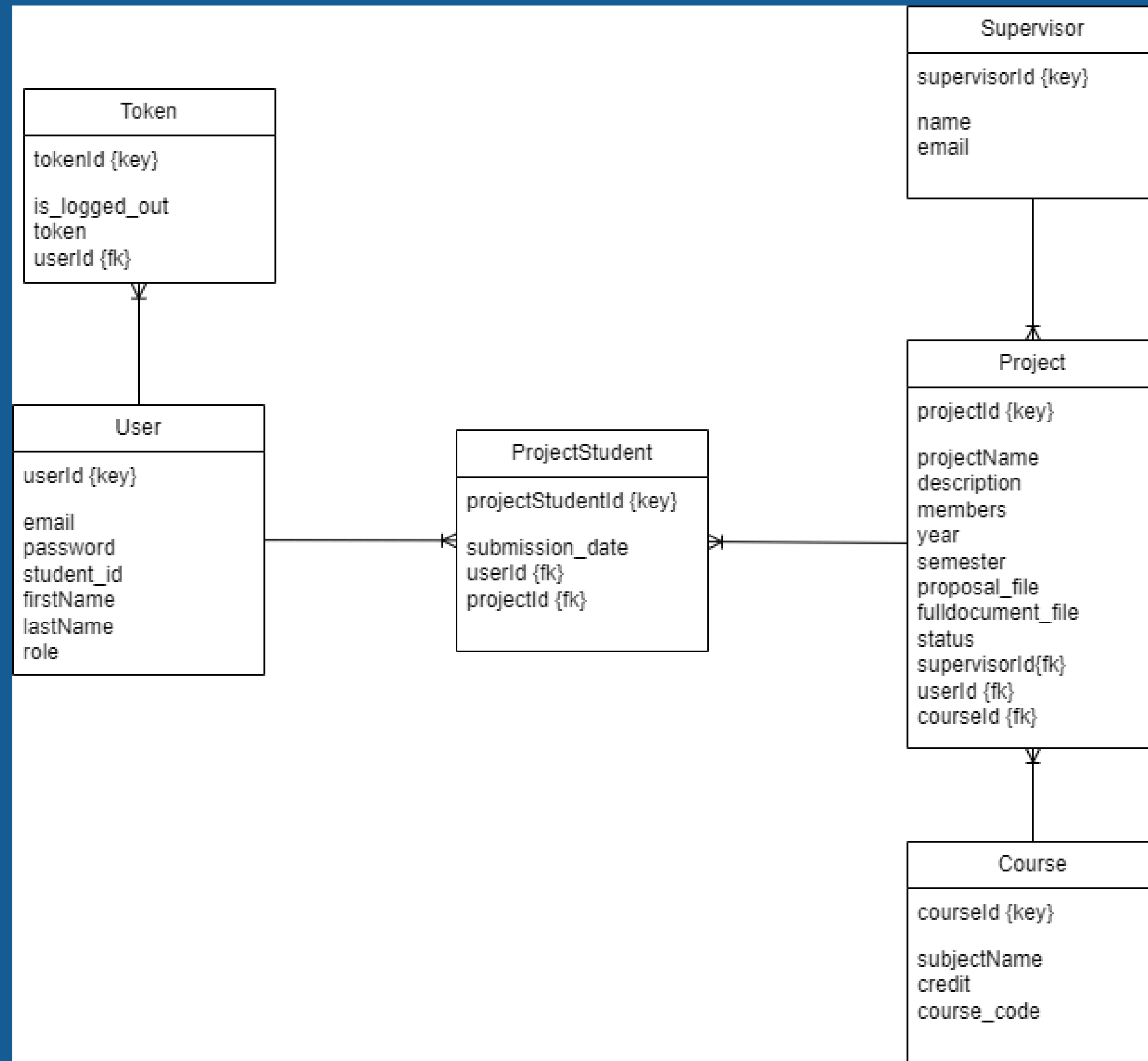
- ReactJS
- Spring boot
- MySQL
- Framework: Bootstrap 5



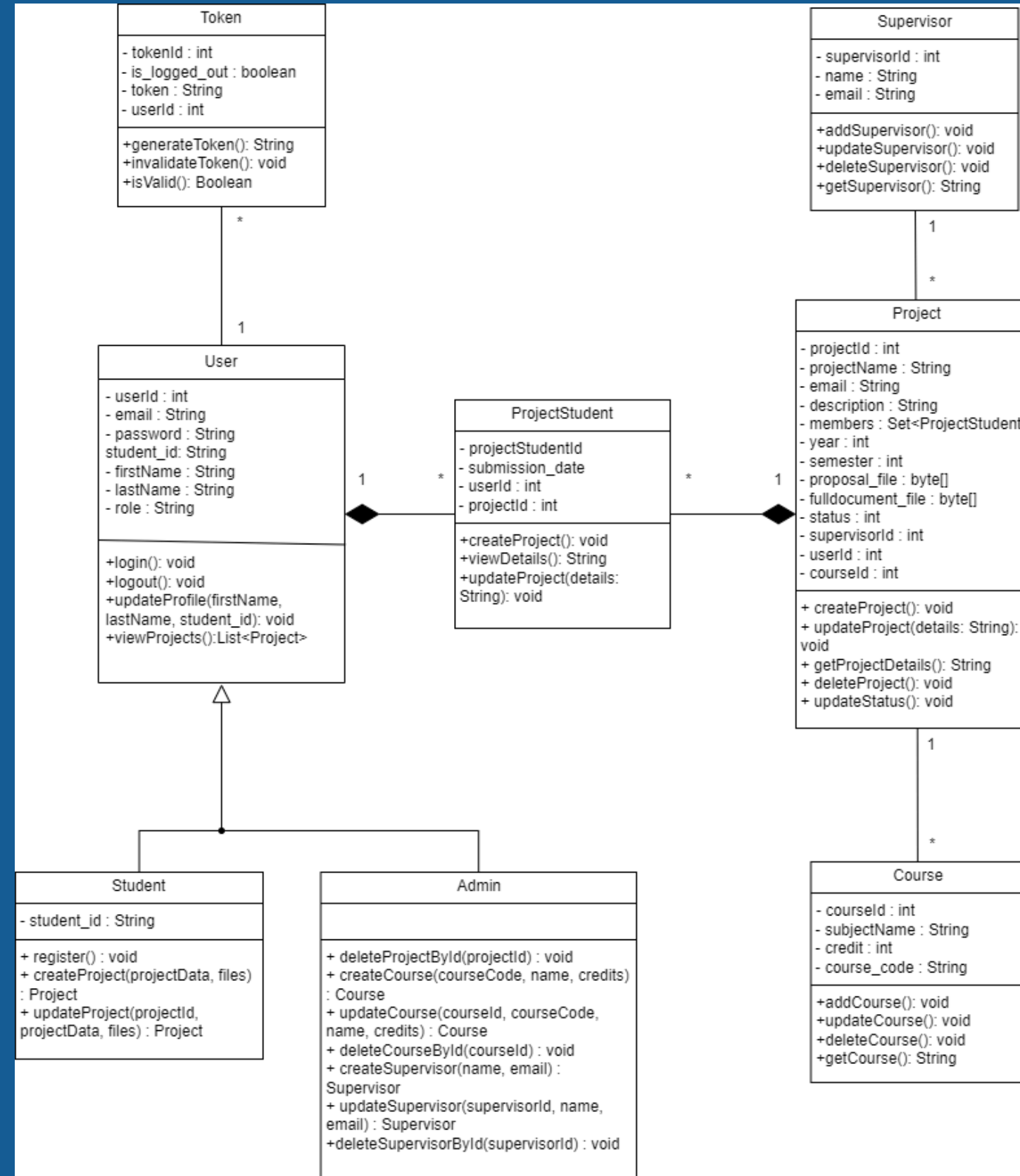
USE CASE DIAGRAM



ERD (ENTITY RELATIONSHIP DIAGRAM)



CALSS DIAGRAM



หลักการ SOLID

SINGLE RESPONSIBILITY PRINCIPLE (SRP)

นำมาใช้ในโค้ดโดยมีการแยกความรับผิดชอบของแต่ละคลาส เช่น **ProjectController** รับผิดชอบจัดการกับ **HTTP Request** และ **Response** **ProjectService** รับผิดชอบใน **Business Logic** ของโปรเจกต์ และ **ProjectRepository** รับผิดชอบในส่วนของการเชื่อมต่อกับฐานข้อมูล

OPEN/CLOSED PRINCIPLE (OCP)

โค้ดควรเปิดรับการขยายแต่ปิดสำหรับการแก้ไข ใช้ในการแยกความรับผิดชอบ เช่น **Service** และ **Repository** สามารถขยายฟังก์ชันใหม่ ๆ ได้โดยไม่ต้องแก้ไขโค้ดเดิม

หลักการ SOLID

LIKOV SUBSTITUTION PRINCIPLE [LSP]

สามารถใช้ **Subclass** แทนที่ **Superclass** ได้โดยไม่ทำให้โปรแกรมทำงานผิดพลาด ในการนำมาใช้ในโค้ดของ **Spring Boot** การใช้ **Interface** เช่น **JpaRepository** ทำให้สามารถเปลี่ยนแปลงการทำงานของ **Class** ได้โดยไม่ต้องแก้ไขการเรียกใช้งาน

INTERFACE SEGREGATION PRINCIPLE [ISP]

ไม่ควรบังคับให้คลาสใด ๆ ต้องพึ่งพาเมธอดที่มันไม่ได้ใช้งาน ในการนำมาใช้ การแยกฟังก์ชันการทำงานออกเป็นหลาย ๆ **Service** เช่น **ProjectService, UserService** ช่วยให้แต่ละ **Service** มีหน้าที่เฉพาะ ไม่ต้องแบกรับภาระที่ไม่จำเป็น

หลักการ SOLID

DEPENDENCY INVERSION PRINCIPLE (DIP)

การใช้ Spring Dependency Injection ในการ Inject
Service และ Repository ทำให้ลดการพึ่งพา Class ที่เป็น
Concrete ลง ทำให้สามารถทดสอบและขยายระบบได้ง่าย
ขึ้น

DESIGN PATTERNS

Controller-Service-Repository Pattern

นำมาใช้ **Controller** จัดการการรับและส่งข้อมูล
จากผู้ใช้ **Service** จัดการ **Business Logic**
Repository จัดการการเชื่อมต่อกับฐานข้อมูล
เพื่อแยกความรับผิดชอบ

Factory Pattern

การสร้าง **Repository**, **Service** และ **Controller**
ผ่าน **Spring Container** โดยไม่ต้องสร้างออบ
เจกต์เอง

Singleton Pattern

ใช้เพื่อให้แน่ใจว่าคลาสหนึ่งจะมี **instance** เดียว
Spring Boot ใช้ **Singleton** เป็นค่าเริ่มต้น
สำหรับ **Bean** ทุกตัว ทำให้แน่ใจว่าแต่ละ
Service มี **instance** เดียวตลอดการทำงาน

Decorator Pattern

การใช้ **Annotations** เช่น **@Transactional**,
@Service, และ **@RestController** ซึ่งทำหน้าที่
เพิ่มฟังก์ชันให้กับคลาสหรือเมธอดโดยไม่
เปลี่ยนโค้ดภายใน

**THANK
YOU VERY
MUCH!**

